AGILE DEVELOPMENT PROCESS AND DEVOPS
LAB FAT

Name: Kevin R Abraham

Reg No: 23MIS0061

Faculty: Dr. BALA GANESH N

Date: 15 April 2026

SET 5

Online Donation System

Problem Statement:

Ensure donations are processed and recorded accurately

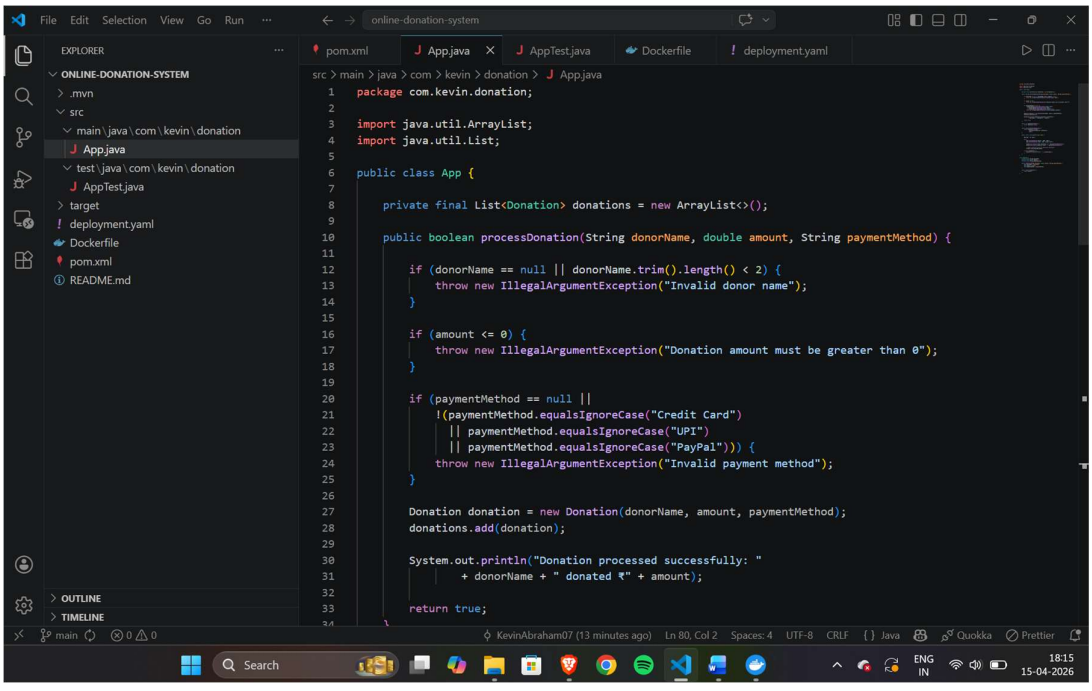
Tasks:

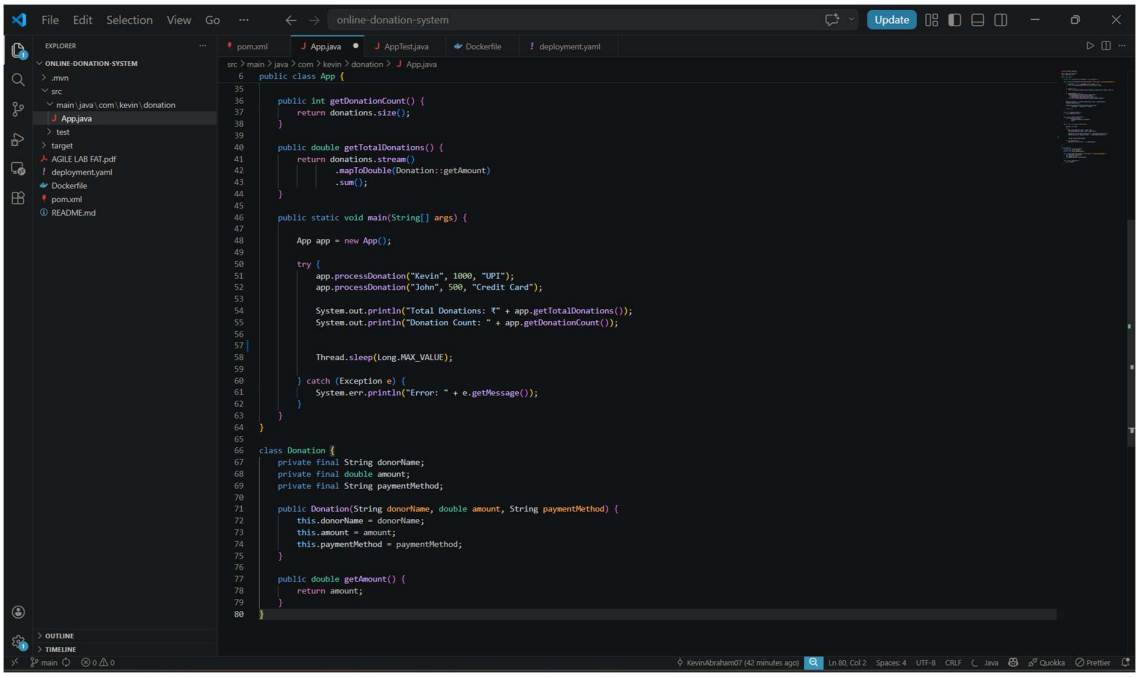
1. Develop donation processing logic
2. Write JUnit tests for transaction validation
3. Implement CI/CD pipeline with automated testing
    - Set up version control using GitHub
    - Configure CI pipeline to trigger on code commits
    - Automate build process using Maven
    - Integrate automated unit tests in pipeline
    - Configure CD pipeline for automatic deployment to staging/production
4. Deploy services on Docker and Kubernetes
    - Containerize application using Docker
    - Build and push Docker image to container registry
    - Create Kubernetes deployment and service YAML files

GitHub Repository Link: https://github.com/KevinAbraham07/Agile-FAT.git

App.java

Contains Java code for Online Donation System

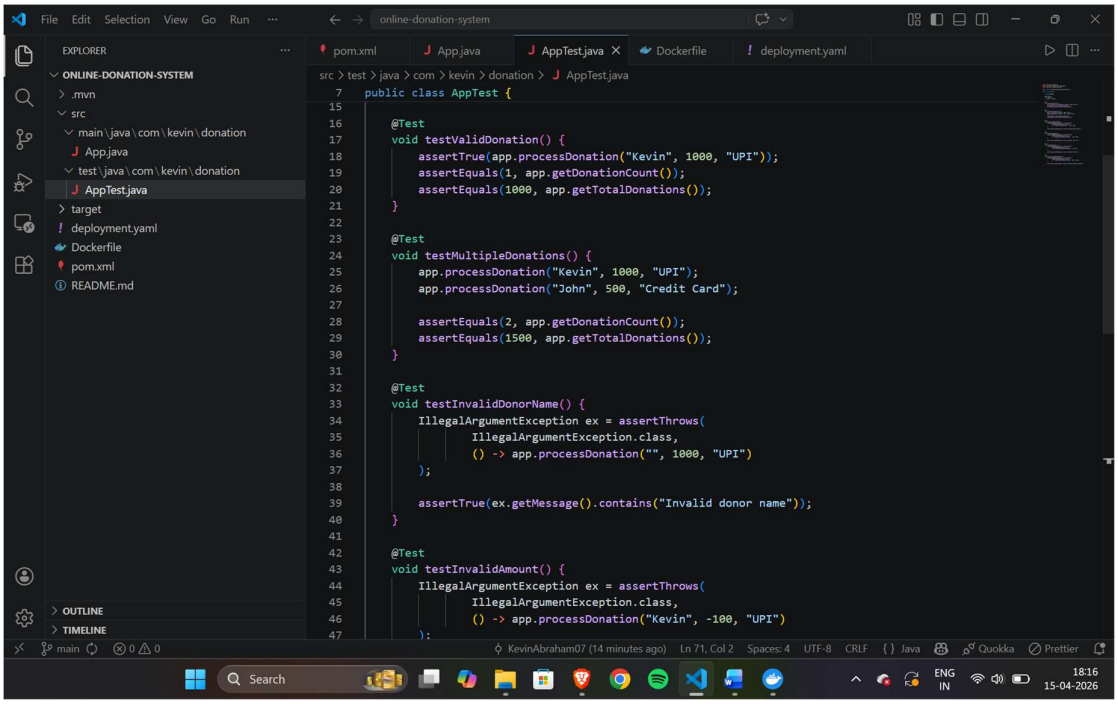




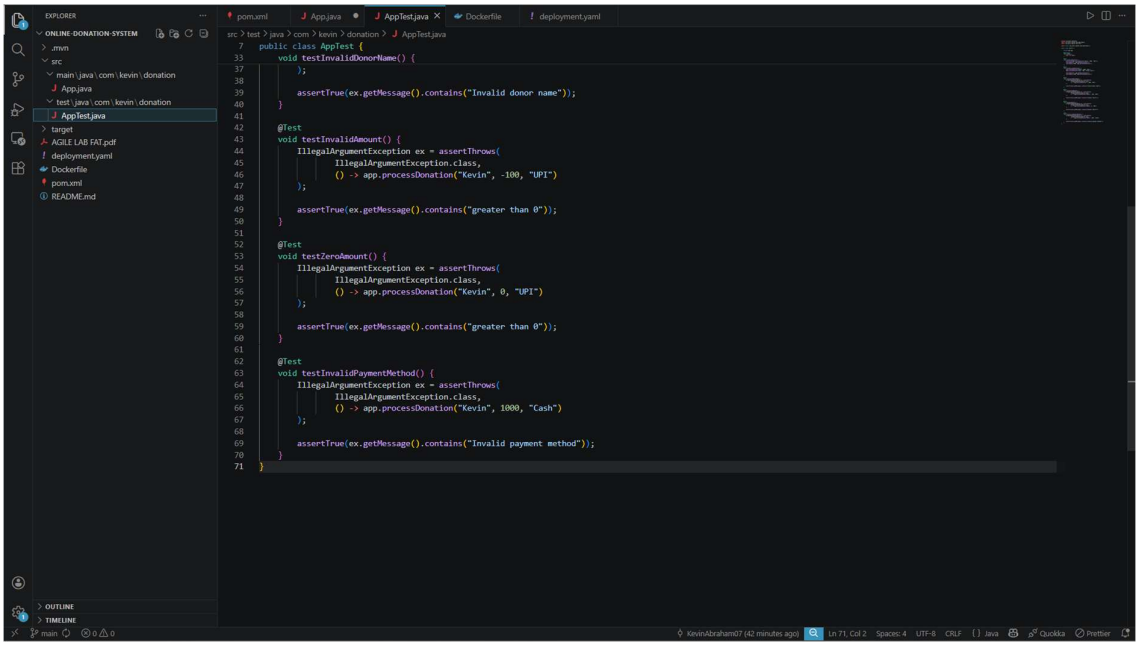
```java
public class App {

    public int getDonationCount() {
        return donations.size();
    }

    public double getTotalDonations() {
        return donations.stream()
                .mapToDouble(Donation::getAmount)
                .sum();
    }

    public static void main(String[] args) {

        App app = new App();

        try {
            app.processDonation("Kevin", 1000, "UPI");
            app.processDonation("John", 500, "Credit Card");

            System.out.println("Total Donations: ₹" + app.getTotalDonations());
            System.out.println("Donation Count: " + app.getDonationCount());


            Thread.sleep(Long.MAX_VALUE);

        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}

class Donation {
    private final String donorName;
    private final double amount;
    private final String paymentMethod;

    public Donation(String donorName, double amount, String paymentMethod) {
        this.donorName = donorName;
        this.amount = amount;
        this.paymentMethod = paymentMethod;
    }

    public double getAmount() {
        return amount;
    }
}
```

AppTest.java

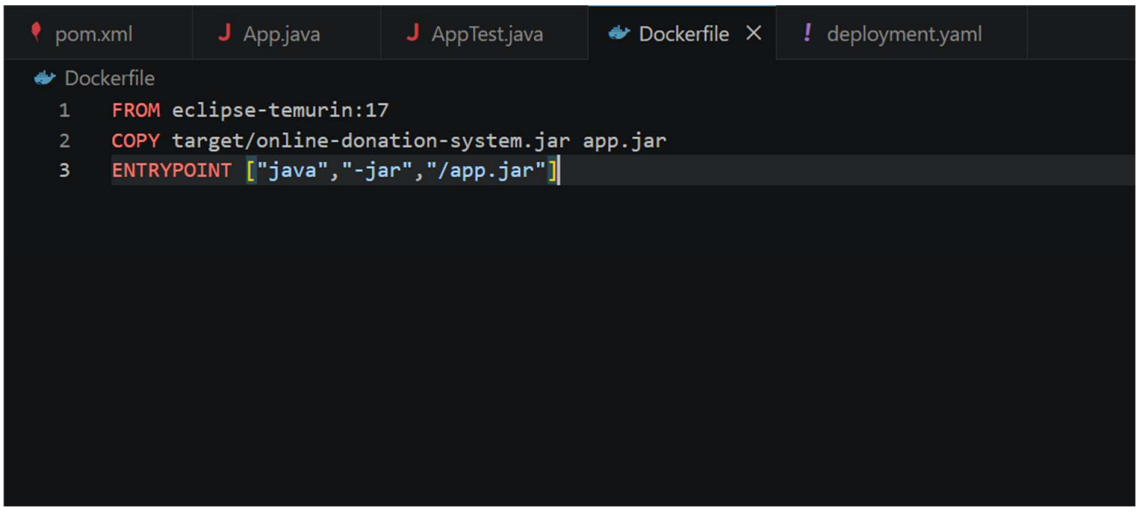
Contains JUnit Testing Code for Online Donation System



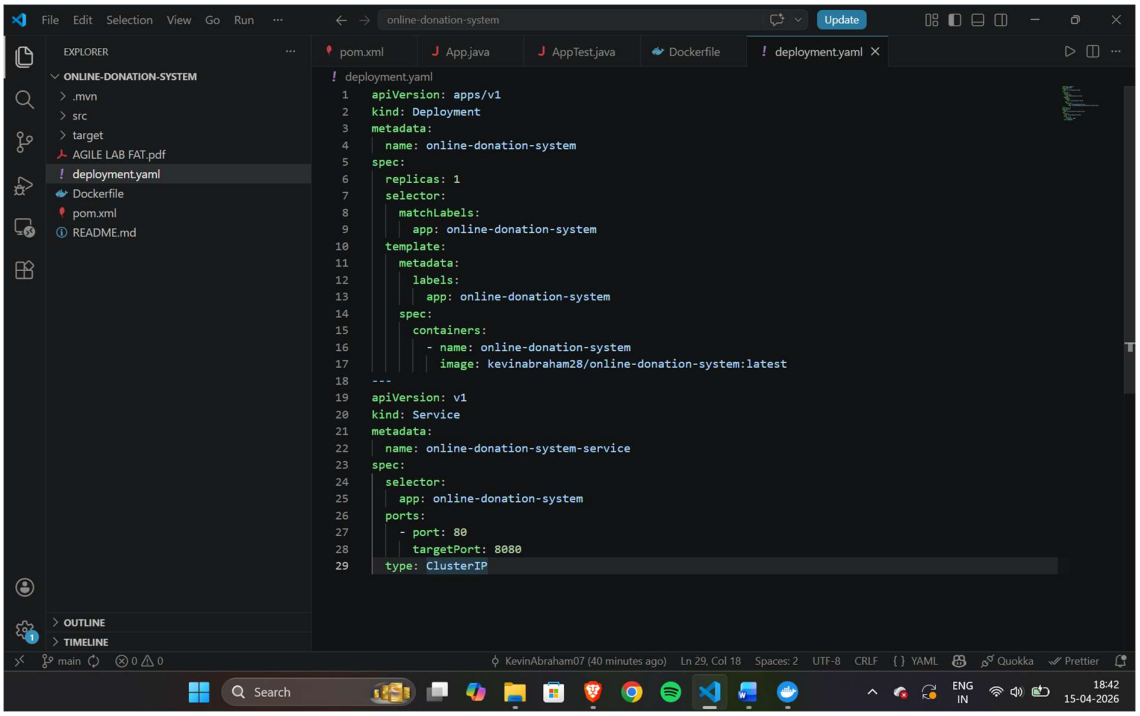
```java
public class AppTest {

    @Test
    void testValidDonation() {
        assertTrue(app.processDonation("Kevin", 1000, "UPI"));
        assertEquals(1, app.getDonationCount());
        assertEquals(1000, app.getTotalDonations());
    }

    @Test
    void testMultipleDonations() {
        app.processDonation("Kevin", 1000, "UPI");
        app.processDonation("John", 500, "Credit Card");

        assertEquals(2, app.getDonationCount());
        assertEquals(1500, app.getTotalDonations());
    }

    @Test
    void testInvalidDonorName() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> app.processDonation("", 1000, "UPI")
        );

        assertTrue(ex.getMessage().contains("Invalid donor name"));
    }

    @Test
    void testInvalidAmount() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> app.processDonation("Kevin", -100, "UPI")
        );
```



Dockerfile

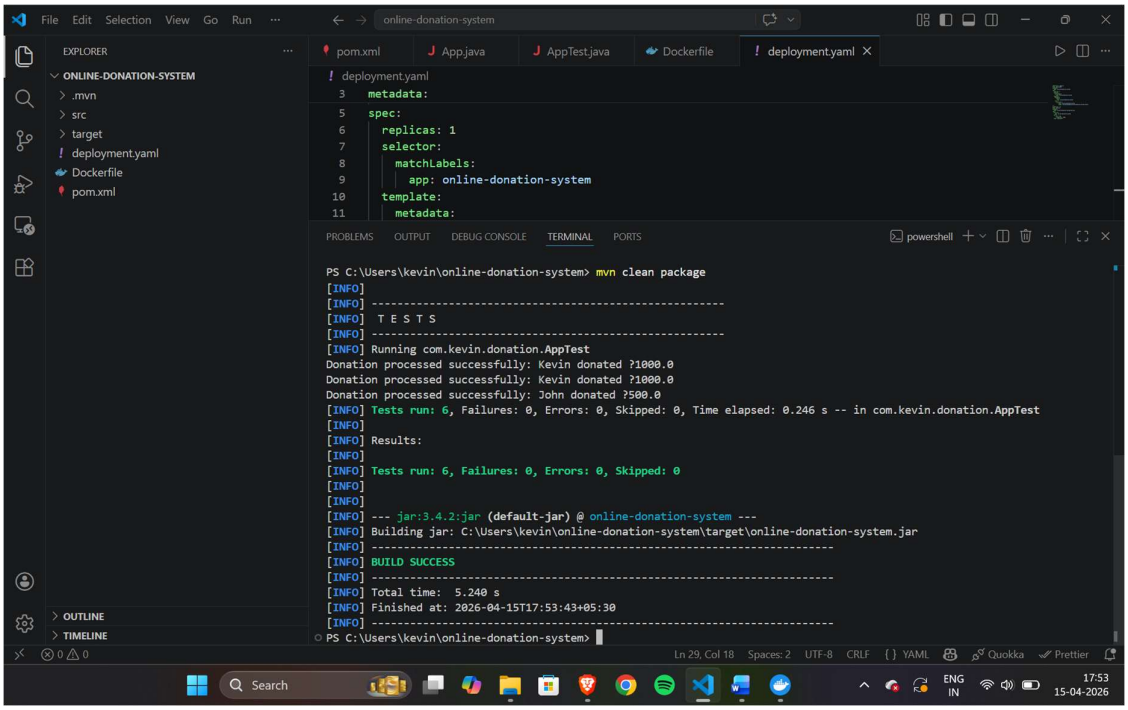


deployment.yaml



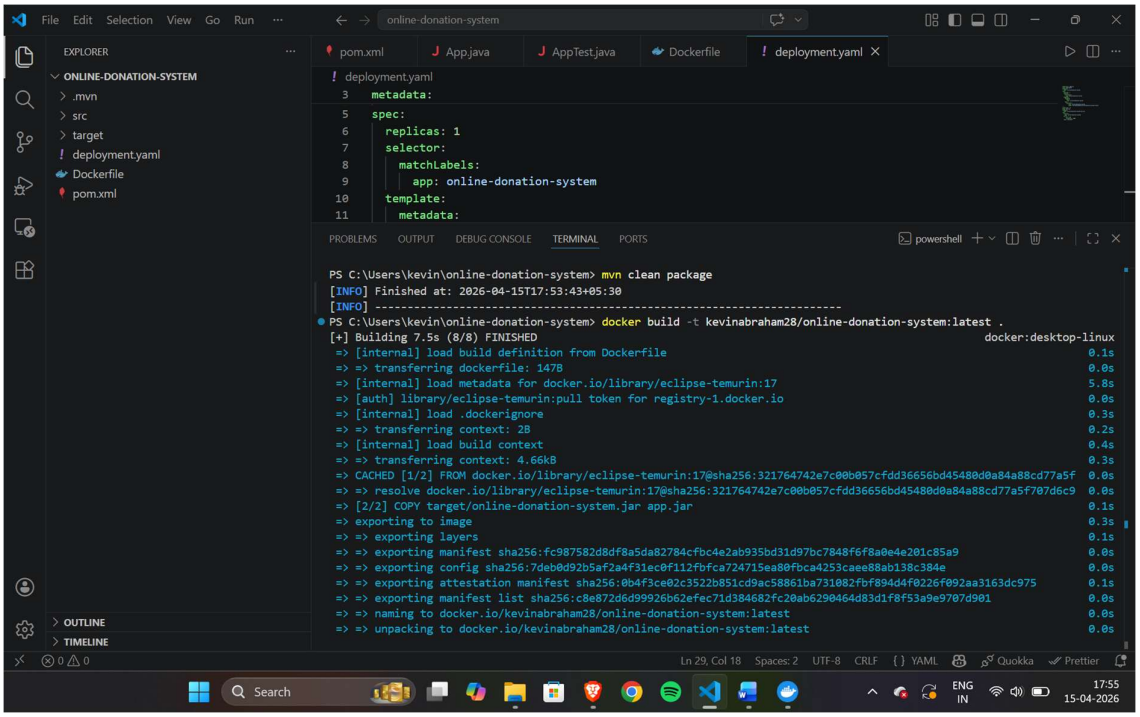
mvn clean package

Compiles the Java application, runs tests, and packages it into an executable JAR file.



docker build -t kevinabraham28/online-donation-system:latest .

Builds a Docker image for the application and tags it for Docker Hub.



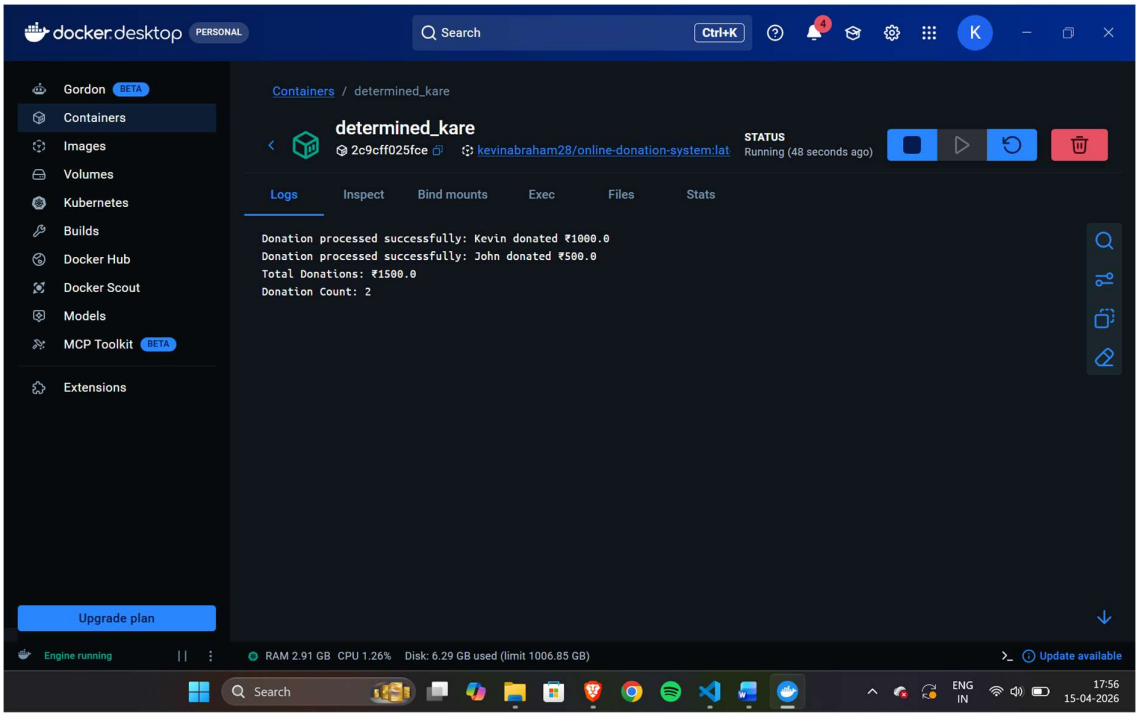
docker run kevinabraham28/online-donation-system:latest

Runs the built Docker image as a container locally for testing.

```
PS C:\Users\kevin\online-donation-system> docker run kevinabraham28/online-donation-system:latest
Donation processed successfully: Kevin donated ₹1000.0
Donation processed successfully: John donated ₹500.0
Total Donations: ₹1500.0
Donation Count: 2
```

New Docker Container Created

A Docker container was successfully created and run from the built image, therefore confirming the creation of cluster



docker push kevinabraham28/event-registration-app:latest

Uploads the Docker image to Docker Hub for remote storage and deployment.

```
PS C:\Users\kevin\online-donation-system> docker push kevinabraham28/online-donation-system:latest
The push refers to repository [docker.io/kevinabraham28/online-donation-system]
a1e6e1a167f1: Mounted from kevinabraham28/todo-management-app
d0061ec7f38e: Pushed
38d0b5bce0f2: Pushed
689b91d88a0f: Mounted from kevinabraham28/todo-management-app
a7428f4f5c1d: Mounted from kevinabraham28/todo-management-app
a149c623348d: Mounted from kevinabraham28/todo-management-app
f00bd8c488bb: Mounted from kevinabraham28/todo-management-app
latest: digest: sha256:c8e872d6d99926b62efec71d384682fc20ab6290464d83d1f8f53a9e9707d901 size: 856
```

kubectl apply -f deployment.yaml

Deploys the application in the Kubernetes cluster using the deployment configuration.

```
PS C:\Users\kevin\online-donation-system> kubectl apply -f deployment.yaml
deployment.apps/online-donation-system created
service/online-donation-system-service created
```

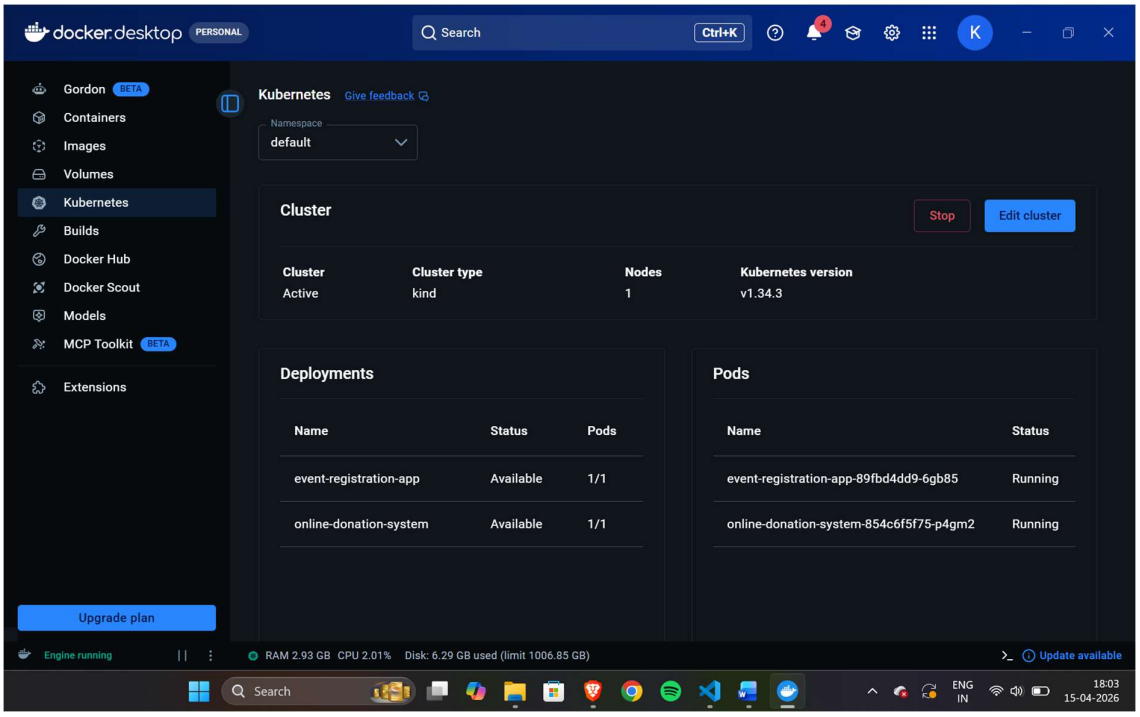
kubectl get pods

Displays the status of all Kubernetes pods in the cluster to verify that the deployed application container is running successfully.

```
PS C:\Users\kevin\online-donation-system> kubectl get pods
NAME                                    READY   STATUS    RESTARTS      AGE
event-registration-app-89fbd4dd9-6gb85  1/1     Running   1 (58m ago)   4d19h
online-donation-system-854c6f5f75-p4gm2 1/1     Running   0             22m
```

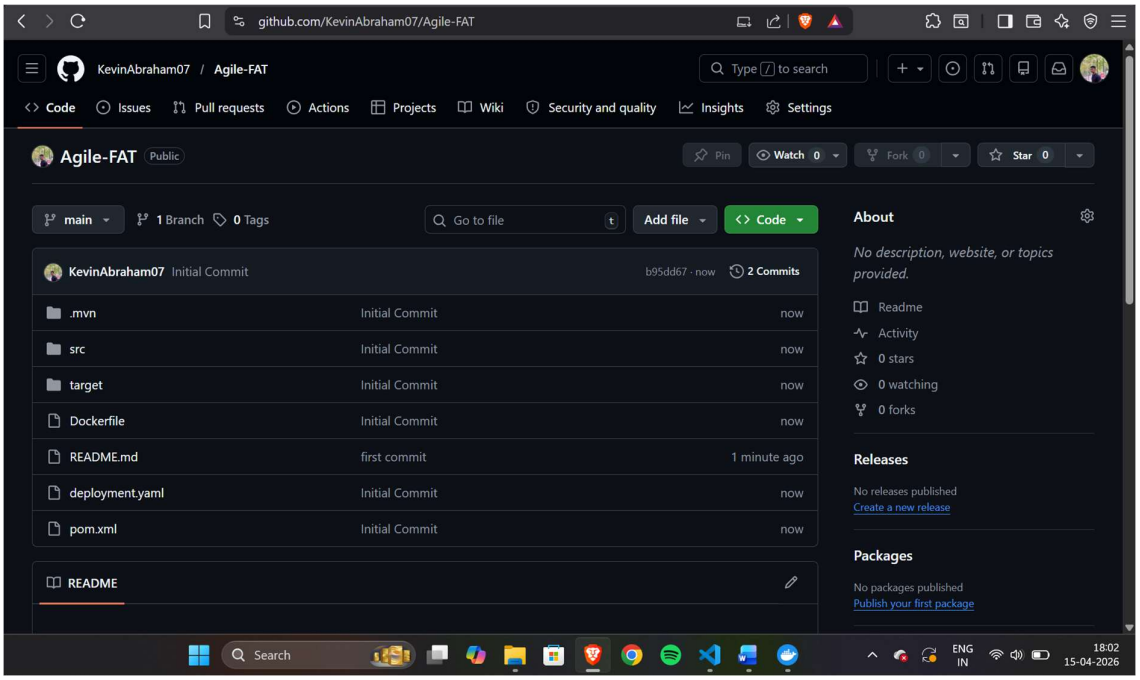
Pushed to Kubernetes Cluster

The application container is deployed to the Kubernetes cluster, where the deployment status is monitored.



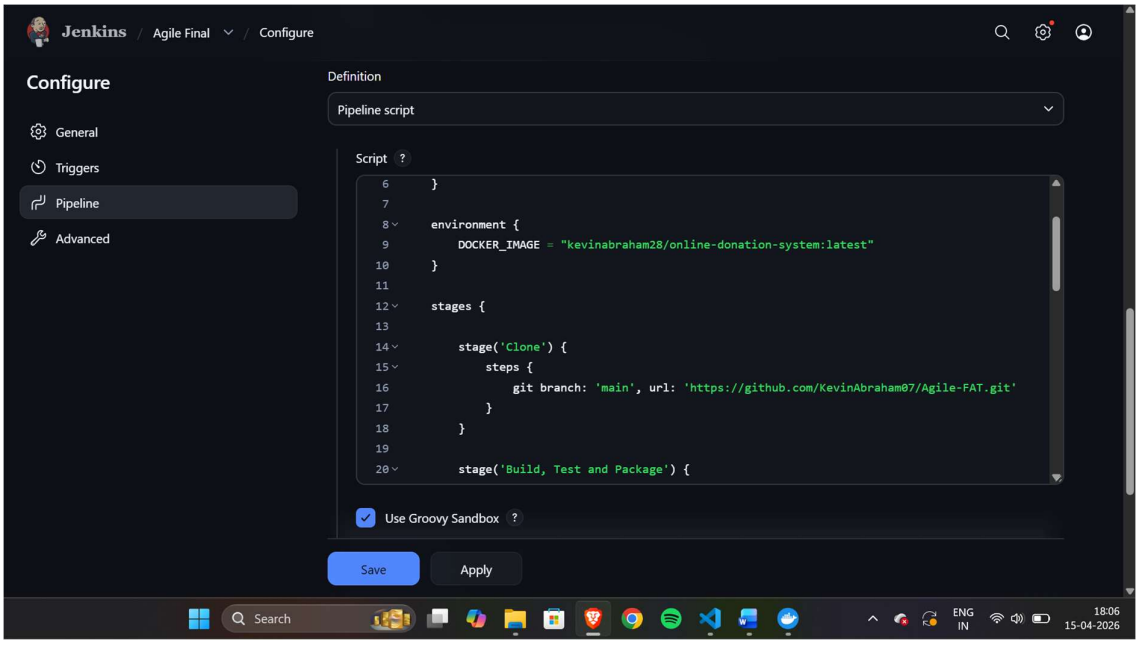
Push to GitHub Repository

The files are uploaded to GitHub to facilitate cloning in Jenkins



Jenkins Pipeline

The Jenkins pipeline configuration automates the CI/CD workflow by cloning the repository, building the project, running tests, containerizing the application, pushing the image, and deploying to Kubernetes.



CI/CD Pipeline Stages

The automated CI/CD pipeline is successfully executed